

Claude Code Sourcemap 项目

Agent 与 LLM 工程深度分析

0. 先说明这份文档怎么读

这份文档会尽量避免“术语堆术语”的写法。

所以你会看到我在第一次提到一些英文词时，顺手把它解释掉：

- **Agent**：不是单纯“大模型”，而是“能持续接任务、用工具、保存状态、继续工作的执行体”
- **Runtime**：运行时，意思是系统真正跑起来时那套机制，不是静态代码结构
- **Query loop**：查询主循环，可以理解为“模型思考一次 -> 系统执行动作 -> 再把结果喂给模型 -> 模型继续思考”的循环
- **Tool calling**：工具调用，模型不是直接回答，而是先发起一个“去读文件/跑命令/搜信息”的动作请求
- **Workflow**：工作流编排，指任务在系统里如何分阶段流动，不只是一个函数调用顺序
- **RAG**：检索增强生成，先“找资料”，再“基于资料回答”
- **Compact**：上下文压缩，把太长的历史对话压成摘要，防止上下文爆掉
- **Attachment**：附加上下文，不一定放在 system prompt 里，而是作为对话中的额外提醒或材料插进去
- **Forked agent**：分叉出来的子执行流，继承主会话的一部分上下文，但不直接污染主线程
- **Coordinator**：协调者 Agent，自己不一定亲自做所有事，而是负责拆任务、派工、收结果、做综合
- **MCP**：Model Context Protocol，一种把外部工具、资源、提示接到模型系统里的标准化协议

如果你一开始觉得这些词很多，没关系。你不要先记词，而是先记住一句话：

这个项目在做的事情，本质上是让模型从“会说话”变成“会长期工作”。

下面所有设计，都是围绕这句话展开的。

1. 总判断：这不是聊天壳，而是一个 Agent Runtime

很多人第一次看这种项目，会觉得它像：

- 一个命令行聊天工具
- 加了一些文件读写能力
- 再加了一点工具调用

但如果你认真看源码，会发现它其实更像一个 **Agent Runtime**，也就是“Agent 运行时系统”。

1.1 什么叫 Runtime

Runtime 这个词，很多人第一次看到会有点抽象。

你可以把它理解成：

- 不是“系统有哪些模块”

- 而是“系统实际工作的时候，这些模块怎样配合”

比如：

- 用户说一句话之后，系统先做什么
- 模型要用工具时，系统怎么判断能不能执行
- 工具执行回来以后，怎么继续让模型推理
- 历史对话太长时，怎么压缩
- 子 Agent 跑出来的结果，怎么回到主 Agent

这些都属于 runtime 的问题。

而 Claude Code 真正厉害的地方，是它不是在做一个“静态工具集合”，而是在做一套“动态工作机制”。

1.2 它到底围绕什么目标构建

我认为它主要围绕四个工程目标：

1. 让模型能持续完成多步任务，而不是只完成一轮问答
2. 让模型能稳定调用工具，而不是偶尔演示一下 function calling
3. 让系统在会话很长、工具很多、Agent 变多的时候还能工作
4. 让知识、规则、偏好能够沉淀，不是每次都从零开始

为了达成这四个目标，它把系统拆成了几个核心部分：

- 会话引擎：负责“这一场对话”怎么持续下去
- Prompt 系统：负责“模型现在到底看到了什么规则和上下文”
- Tool Runtime：负责“模型想做动作时，系统怎么替它执行”
- Memory 系统：负责“什么东西值得留下来，以后再用”
- Multi-Agent 系统：负责“一个模型不够时，怎么组织多个执行体协作”

这也是为什么我说它不是一个聊天壳。

聊天壳只关心：

- 用户输入什么
- 模型输出什么

而这个项目关心的是：

- 模型工作过程中会发生什么

2. 整体架构：它分了哪些层，每层在解决什么问题

如果要把整个系统讲清楚，我会把它分成七层。这里我不用太工程化的方式讲，而用更直白的话讲。

2.1 入口与环境层

关键文件：

- `restored-src/src/main.tsx`

这一层负责的是：

- 系统怎么启动
- 配置从哪里读
- 用户是谁
- 有哪些插件、技能、MCP 服务可用
- 当前会话属于什么模式

你可以把它理解成“开机层”。

为什么这层重要？

因为很多看起来是“模型能力”的东西，其实根本不是模型决定的，而是启动阶段就决定的，比如：

- 当前有没有登录
- 是否启用了某个 feature gate（功能开关）
- 允许不允许某些工具
- 当前有没有接入 MCP server
- 当前是不是 coordinator 模式

也就是说，模型看到的世界，不是天然存在的，而是启动层先搭出来的。

这就像一个人上工之前，先得知道：

- 我今天在哪个办公室
- 我能进哪些房间
- 我有什么工具箱
- 我今天是普通员工还是项目经理

2.2 会话引擎层

关键文件：

- `restored-src/src/QueryEngine.ts`

这一层负责的是：

- 这一场会话现在处于什么状态
- 历史消息有哪些
- 已经花了多少 token
- 读过哪些文件
- 当前有哪些 memory 被注入过

这层的核心对象就是 `QueryEngine`。

如果你不熟悉这种写法，可以把它想成：

- 它不是“发一次请求”
- 它是“这场对话的总管”

为什么这层很关键？

因为真正的 Agent 对话不是：

- 一次输入，一次输出

而是：

- 用户说一句
- 模型读文件
- 模型改文件
- 模型再问工具
- 用户再插一句
- 系统再 compact 一次

如果没有一个持续存在的“会话对象”，这些状态就会散落在一堆函数里，后面很容易失控。

2.3 推理与执行循环层

关键文件：

- `restored-src/src/query.ts`

这是全系统最值得盯住的一层。

这里的 `query loop`，你可以把它想象成下面这个循环：

1. 系统先给模型准备上下文
2. 模型回答，或者提出工具调用
3. 系统执行工具
4. 工具结果回给模型
5. 模型继续推理
6. 直到这一轮任务结束

这就是 `query loop`。

很多人第一次接触 Agent 时，会把 Agent 理解成：

- “模型+工具”

但真正更准确的说法应该是：

- “模型在 `query loop` 里反复思考和行动”

为什么这很重要？

因为一旦你理解了 this 循环，你就会发现：

- Prompt 不是一开始写完就结束

- Tool calling 不是外挂
- compact 也不是边角功能

它们都在这个循环里影响模型下一步怎么想。

2.4 Prompt 与上下文层

关键文件：

- `restored-src/src/constants/prompts.ts`
- `restored-src/src/utls/systemPrompt.ts`
- `restored-src/src/constants/systemPromptSections.ts`
- `restored-src/src/utls/attachments.ts`

这一层负责的不是“写一段漂亮提示词”，而是：

- 让模型每一轮都看到恰到好处的规则和上下文

注意这里有两个很重要的词：

- `System prompt`：系统提示词，相当于长期工作守则
- `Attachment`：附加上下文，相当于这一轮临时要提醒它看的材料

这个项目里，很多人初看会疑惑：

- 为什么不把东西都塞进 system prompt?

答案是：

- 因为很多信息是会变化的
- 一变化就会让缓存失效
- 所以要把“稳定规则”和“动态信息”拆开

后面我会专门讲。

2.5 Tool Runtime 层

关键文件：

- `restored-src/src/Tool.ts`
- `restored-src/src/services/tools/toolOrchestration.ts`
- `restored-src/src/services/mcp/client.ts`

这一层负责的是：

- 模型发起的工具请求怎么真正落地执行

请注意，这里不是简单的“函数调用”。

真正发生的事情是：

- 校验参数
- 检查权限

- 判断能否并发
- 执行工具
- 采集结果
- 把结果包装成消息
- 继续回到 query loop

也就是说，这是一层“工具执行操作系统”，不是函数列表。

2.6 记忆与知识层

关键文件：

- `restored-src/src/memdir/*`
- `restored-src/src/utils/claudemd.ts`
- `restored-src/src/services/extractMemories/*`
- `restored-src/src/services/autoDream/*`

这层负责：

- 什么东西应该留下来
- 怎么留
- 以后怎么再找到它

你可以把这层分成两类：

- **规则型知识**：项目规则、用户约定、团队规范
- **经验型记忆**：这个用户喜欢什么、这个项目有什么坑、之前发生过什么

2.7 多 Agent 协作层

关键文件：

- `restored-src/src/tools/AgentTool/*`
- `restored-src/src/coordinator/coordinatorMode.ts`
- `restored-src/src/utils/swarm/*`

这一层负责：

- 当一个 Agent 不够时，怎么把任务拆给多个执行体

这里最重要的不是“能 spawn 子 Agent”，而是：

- 谁拆任务
- 谁干活
- 谁收结果
- 谁对用户负责

这其实已经不是单纯的软件问题，而是组织问题。

2.8 如果你想真正读懂源码，最推荐的“主链路阅读法”

前面我把系统拆成了七层，但你如果真的去读源码，不能按“名词分类”去读。

因为源码不是百科全书，不会把 Prompt、Tool、Memory、Workflow 自动分章节摆好给你。

更有效的读法，是顺着一条真实请求，把关键控制点串起来读。

我最推荐的顺序是：

1. `restored-src/src/main.tsx`
2. `restored-src/src/QueryEngine.ts`
3. `restored-src/src/utils/queryContext.ts`
4. `restored-src/src/utils/systemPrompt.ts`
5. `restored-src/src/utils/processUserInput/processUserInput.ts`
6. `restored-src/src/query.ts`
7. `restored-src/src/services/tools/toolOrchestration.ts`
8. `restored-src/src/query/stopHooks.ts`
9. `restored-src/src/services/extractMemories/extractMemories.ts`
10. `restored-src/src/services/autoDream/autoDream.ts`
11. `restored-src/src/tools/AgentTool/runAgent.ts`
12. `restored-src/src/utils/swarm/inProcessRunner.ts`

为什么要按这个顺序？

因为它基本对应了一个完整 Agent Runtime 的真实执行链：

- `main.tsx` 决定环境怎么装起来
- `QueryEngine.ts` 决定一轮请求怎么入场、会话状态怎么持有
- `queryContext.ts` 和 `systemPrompt.ts` 决定模型在这一轮到底看到什么
- `processUserInput.ts` 决定用户输入在进入推理前被如何解释
- `query.ts` 决定主循环怎样跑
- `toolOrchestration.ts` 决定模型动作怎样落地
- `stopHooks.ts` 决定这一轮结束后有哪些后台治理动作
- `extractMemories.ts` / `autoDream.ts` 决定会话怎样沉淀成长期资产
- `runAgent.ts` / `inProcessRunner.ts` 决定单 Agent 之外的子 Agent、多 Agent 怎样工作

你会发现，这其实不是“从文件 A 到文件 B”的随便跳转，而是一条很明确的工程主线：

环境装配 -> 会话建立 -> 上下文供给 -> 推理循环 -> 动作执行 -> 后处理治理 -> 长期沉淀 -> 多执行体扩展

这条主线非常重要，因为很多人读这种项目，最容易犯的错误是：

- 一上来钻到某个花哨工具里
- 或者一上来研究某个很长的 prompt
- 或者只看 AgentTool，却不看主会话链路

结果就是局部看了很多，整体却始终没有建立起来。

所以如果你真想从这个项目里学东西，最应该学会的能力不是“记住每个函数干什么”，而是：

看见一个复杂系统时，先找到它的主执行链，再把其他机制挂回主链上理解。

这也是为什么我前面一直强调，这个项目的核心不是某一个 Prompt、某一个 Tool、某一个 Agent，而是它把这些东西都挂在同一个 Runtime 里。

3. Agent 工程：这里的 Agent 究竟分几种，它们分别干什么

很多文章一提 Agent，只会说：

- “Agent 就是会调用工具的大模型”

这句话不能说错，但太粗了。在这个项目里，Agent 实际上分了好几层，每一层的职责不同。

3.1 主 Agent：直接面对用户的那个执行体

对应位置：

- `QueryEngine.ts`
- `query.ts`

主 Agent 负责：

- 理解用户需求
- 决定要不要用工具
- 决定要不要派生子 Agent
- 最后把结果说给用户

为什么要强调“主 Agent”？

因为在多 Agent 系统里，不是每个 Agent 都一样。

主 Agent 最重要的职责，不是“做最多事”，而是：

- 负责最终理解
- 负责最终汇总
- 负责对用户说最终的话

如果这一层不稳，就会出现一种常见混乱：

- 子 Agent 干了很多
- 但没人真正综合
- 最后用户得到的是拼贴结果，不是清晰结论

3.2 Forked Agent：为了做侧任务而分叉出去的执行流

对应位置：

- `restored-src/src/utils/forkedAgent.ts`

这个词很容易让人困惑，所以我先用最通俗的话解释：

`Forked agent` 就像你自己工作到一半，复制出一个“临时副本自己”去旁边查点事，查完回来告诉你结果，但它不要把中间过程全都带回来。

它解决的核心问题是：

- 有些工作值得单独跑
- 但不值得污染主线程上下文

比如：

- 抽取 memory
- 做会话总结
- 跑一个 side query

为什么不直接主线程做？

因为主线程上下文很贵，而且主线程要保持清晰。

更关键的是，这个项目里的 fork 还考虑了 `prompt cache`（提示词缓存）问题。

3.3 什么是 Prompt Cache，为什么这里要专门考虑

很多模型服务会对一部分“重复前缀”做缓存。

比如：

- 相同的 system prompt
- 相同的大段工具描述
- 相同的大段上文

下一次再发时，成本和延迟可以下降。

`utils/forkedAgent.ts` 里定义了 `CacheSafeParams`，意思是：

- 哪些参数必须和父请求保持一致
- 才能最大化复用已有缓存

这背后非常体现工程成熟度，因为它关注的不只是“逻辑能不能跑”，还关注：

- 成本能不能压住
- 延迟能不能可控

很多系统做到后期才发现，最大的瓶颈不是模型能力，而是：

- 每一轮都太贵
- 每一个 side task 都太慢

这里其实是在提前做“成本工程”。

3.4 Subagent：有角色、有能力边界的专职 Agent

对应位置：

- `restored-src/src/tools/AgentTool/loadAgentsDir.ts`
- `restored-src/src/tools/AgentTool/runAgent.ts`

这个项目里的 Agent 配置远不止一段 prompt，它包括：

- `prompt`：这个 Agent 的系统说明
- `tools / disallowedTools`：它能用什么，不能用什么
- `model`：用哪个模型
- `effort`：推理强度
- `permissionMode`：权限策略
- `mcpServers`：依赖哪些外部 MCP 服务
- `memory`：使用哪一类持久记忆
- `background`：是不是后台任务
- `isolation`：要不要运行在隔离环境

这件事非常值得你学。

因为很多人理解 Agent 时只想到：

- 换一个 Prompt

但真正的工业 Agent，差异不只是 Prompt，而是：

- 能力边界
- 权限边界
- 运行边界

也就是说，Agent 的本质不只是 persona（人格），更是 envelope（能力边界包络）。

3.5 Coordinator：协调型 Agent

对应位置：

- `restored-src/src/coordinator/coordinatorMode.ts`

这是我认为整个项目最体现“系统设计水平”的模块之一。

为什么？

因为它不是简单说一句：

- 你可以开几个子 Agent 并行工作

而是明确规定了多 Agent 系统中的角色分工：

- 哪些事情适合并行研究
- 哪些事情要由 coordinator 自己做综合
- 哪些写操作不能并行

- 验证应该如何执行
- worker 返回结果时, coordinator 应该怎么理解, 怎么再派工

这里的 `Workflow` (工作流) 不再是函数调用顺序, 而更像一个小组织的工作方式。

如果你把主 Agent 当成“项目负责人”, 把 worker 当成“执行成员”, 这个模块就好理解了。

3.6 Swarm / teammate: 再往下一层的多 Agent 运行机制

对应位置:

- `restored-src/src/utils/swarm/inProcessRunner.ts`
- `restored-src/src/utils/swarm/*`

这层更底层, 解决的是:

- 多个 Agent 在哪里跑
- 怎么通信
- 怎么请求权限
- 怎么把消息回到主线程

这里最值得注意的是:

子 Agent 不是“自己随便跑”, 而是也受一个基础设施系统管理。

比如:

- ask 权限时, 要桥接到 leader
- 子 Agent 自己也可能 compact
- 子 Agent 的状态要进 task system

这说明作者已经意识到:

多 Agent 的难点不是“多开几个模型”, 而是“多份运行时状态如何组织”。

3.7 如果不这样分层做 Agent, 会有哪些更简单但更脆弱的方案

为了让你真正理解“为什么这里把 Agent 拆这么细”, 我把几种更常见的简单方案也列出来。

方案 A: 永远只有一个 Agent

这是最省事的方案:

- 所有事情都交给主 Agent
- 不拆 side task
- 不开子 Agent

优点:

- 架构简单
- 调试最直接

缺点:

- 上下文会快速膨胀
- 所有中间噪声都污染主线程
- 很难并行处理复杂任务

什么时候它仍然够用？

- 任务短
- 工具少
- 不需要多线程思考

什么时候它一定开始不够？

- 你开始做长任务
- 你开始做研究、实现、验证分离
- 你希望某些分析在后台跑而不打扰主线程

方案 B：有子 Agent，但所有子 Agent 都和主 Agent 一个模子

这是第二种常见做法：

- 子 Agent 只是“同样模型再开一份”
- 没有角色区分
- 没有工具边界
- 没有权限差异

优点：

- 比单 Agent 强一点
- 不需要复杂 agent spec

缺点：

- 子 Agent 不知道自己职责边界
- 权限和工具范围不清
- 结果风格和行为模式容易漂

为什么 Claude Code 要引入 agent definitions？

因为它已经不满足于“有个子线程模型”，而是希望：

- 某些 Agent 做研究
- 某些 Agent 做验证
- 某些 Agent 用特定工具
- 某些 Agent 继承某类 memory

也就是说，它把 Agent 当成“岗位”来定义，不只是“副本”。

方案 C：有多个 Agent，但没有 coordinator

这也是一个很容易出现的阶段：

- 你能起多个 Agent
- 它们各自工作
- 但没有一个明确负责综合的人

这会带来什么问题？

- 结果四散
- 多个 Agent 可能重复做同样工作
- 没人判断哪个结果更可信
- 用户体验会很差，因为最后像是收到了几段散装报告

这也是为什么 coordinator 很关键。它的意义不只是“调度”，而是：

- 保持任务叙事连贯
- 把多份结果变成一个用户能吸收的结论

3.8 Agent 分层背后的真正设计原则

如果把这一节再往上抽象一下，背后其实有三条原则。

原则一：不同 Agent 不该只在 Prompt 上不同

成熟系统里，Agent 的差异至少还包括：

- 工具范围
- 权限范围
- 记忆范围
- 运行环境
- 角色职责

所以如果你未来自己设计 Agent，不要只问：

- “它的 prompt 是什么？”

还要问：

- “它被允许做什么？”
- “它不该做什么？”
- “它是负责产出结论，还是负责产出材料？”

原则二：主 Agent 应该尽量保持清晰

很多复杂任务里，真正贵的不是一次调用，而是：

- 主线程上下文的清晰度

一旦主线程塞满中间过程、调试碎片、旁支探索，你会发现主 Agent 反而越来越笨。

所以 forked agent、background task、worker 的意义，很大一部分就是：

- 为主线“减负”

原则三：多 Agent 首先是组织问题，其次才是模型问题

你以后如果真的要做多 Agent 系统，最该关注的不是：

- 模型能不能开三个实例

而是：

- 谁负责理解
- 谁负责执行
- 谁负责验证
- 谁负责综合
- 谁对用户负责

这五个问题想不清楚，多 Agent 往往越做越乱。

3.9 如果你自己设计类似系统，Agent 这一层应该怎么选

这里给你一个很实用的判断路径。

如果你现在还是早期阶段

建议：

- 先只做主 Agent
- 最多再加一种简单 fork，用来做 side summary 或 side search

不要急着做：

- coordinator
- swarm
- 大量专职 Agent

因为这会让复杂度过早爆炸。

如果你已经进入中期阶段

你会开始需要：

- 角色化 subagent
- 明确的 tool scope
- 明确的 permission scope

这时就适合做：

- agent definitions
- runAgent 统一执行路径

如果你已经进入复杂任务协作阶段

这时才适合做：

- coordinator
- worker
- task notifications
- mailbox
- permission bridge

也就是说，Agent 系统不是一口气“全做出来”的，而是随着复杂度一步步长出来的。

4. Prompt 设计：为什么这个项目的 Prompt 不是一整段话，而是一套系统

这一部分如果只看表面，很容易觉得：

- “不就是 prompt 很长吗？”

但它真正的设计重点，不是“长”，而是“分层、缓存、动态化”。

4.1 Prompt 的第一目标：行为约束，而不是说话风格

对应位置：

- `restored-src/src/constants/prompts.ts`

很多初学者学 Prompt，会优先关注：

- 语气
- 人设
- 输出格式

但在这个项目里，真正重要的 Prompt 内容是：

- 做软件工程时的工作原则
- 工具使用规范
- 安全边界
- 权限被拒绝时怎么办
- 什么情况下该 compact
- 什么能写入 memory，什么不能写

也就是说，这里的 Prompt 更像：

- 工作守则
- 行为协议
- 风险约束说明书

这对你有个很重要的启发：

当 Agent 真的开始干活时，Prompt 的主要作用不是“更像人”，而是“更像一个靠谱的系统成员”。

4.2 Prompt 的第二目标：可缓存

对应位置：

- `restored-src/src/constants/prompts.ts`
- `restored-src/src/constants/systemPromptSections.ts`

这里有一个非常关键的概念：

- `SYSTEM_PROMPT_DYNAMIC_BOUNDARY`

这个名字看起来有点技术味，但意思其实不难：

- `Boundary` 是边界
- `Dynamic` 是动态变化的

它代表：

- Prompt 前面一部分尽量保持稳定
- 后面一部分允许动态变化

为什么要这样？

因为越稳定的 Prompt 前缀，越容易让缓存生效。

如果你每一轮都把：

- agent 列表
- MCP 指令
- 当前环境
- 当前模式

全部混在 system prompt 正文里，那么只要其中任意一点变化，前缀就会变化，缓存价值就大幅下降。

4.3 `systemPromptSection()` 是什么

这是一个很值得学的工程技巧。

它的意思可以通俗理解成：

- “Prompt 不再是一整块字符串，而是一个个 section（段）”

每个 section 可以有自己的特性：

- 能不能缓存
- 什么时候重新算
- 是否依赖运行时状态

这让 Prompt 设计从“写文案”变成“管理组件”。

4.4 为什么很多动态信息不放 system prompt，而放 attachment

对应位置：

- `restored-src/src/utils/attachments.ts`

这里的 `attachment` 可以理解成：

- 对话流里的附加材料
- 临时提醒
- 补充上下文

比如：

- `skill_discovery`
- `nested_memory`
- `mcp_instructions_delta`
- `plan_mode_exit`

为什么不用 system prompt 全包？

因为 system prompt 更适合：

- 长期稳定规则

attachment 更适合：

- 本轮临时变化信息
- 会不断刷新或重注入的信息

这相当于现实里的区别：

- 员工手册：system prompt
- 今天这次会议的议程和提醒：attachment

4.5 `buildEffectiveSystemPrompt()` 在干什么

对应位置：

- `restored-src/src/utils/systemPrompt.ts`

这个函数名字很直白：

- `Effective`：最终生效的

它负责把不同来源的提示组合起来，比如：

- 默认 prompt
- 某个 agent 的 prompt
- coordinator 模式 prompt
- 用户自定义 prompt
- append prompt

为什么这一步重要？

因为真实系统里，Prompt 很少只有一个来源。

如果没有清晰的优先级规则，你很快就会陷入混乱：

- 到底是默认 prompt 生效，还是 agent prompt 生效？
- append prompt 是覆盖还是追加？
- coordinator 模式时，普通 prompt 还算不算？

所以 Prompt 设计做到最后，其实是在做：

- 提示词优先级管理

4.6 如果不用这种 Prompt 系统，会有哪些常见的“简化版”

这一节我也用对比的方式帮你理解。

方案 A：一个超长 system prompt 打天下

这是最常见的早期做法：

- 所有身份、规则、上下文、注意事项、工具说明
- 全塞进一段超长 system prompt

优点：

- 实现很简单
- 逻辑直观

缺点：

- 越写越长
- 很难维护优先级
- 一部分内容变化就会导致整体变化
- 缓存价值很差

什么时候它还够用？

- 工具少
- 模式少
- 会话不长

什么时候它会迅速失控？

- 你开始有多种 agent 模式
- 开始接入动态 MCP
- 开始接入 memory、skills、attachments

方案 B：所有动态信息都在代码里拼到用户输入前

有些系统会做成：

- system prompt 很短
- 所有当前环境、规则、提醒都作为普通 user content 拼进去

优点：

- 实现容易
- 不需要太复杂的 system prompt 结构

缺点：

- 角色边界混乱
- 模型不一定稳定地区分“系统规则”和“当前材料”
- 规则约束力通常比真正的 system prompt 弱

Claude Code 没这么做，是因为它很明确地区分：

- 什么是长期规则
- 什么是本轮材料

方案 C：所有动态材料也都塞进 system prompt

这和方案 A 类似，但更具体一点：

- agent list 更新也放 system prompt
- MCP instructions 更新也放 system prompt
- 当前模式提醒也放 system prompt

结果就是：

- 只要这些动态信息稍有变化
- 前缀缓存就持续被打爆

所以你现在能更好理解 attachment 的价值：

- 它不是“可有可无的补丁”
- 它是在替 system prompt 减负

4.7 Prompt 设计真正难的，不是写得强，而是写得“稳定”

这里我想再往深一层讲。

很多 Prompt 讨论都在讲：

- 怎么写更强
- 怎么让模型更聪明

但一旦进入产品化，你真正最难的问题往往变成：

- 怎么让 Prompt 在多轮、多模式、多配置下依然稳定

所谓 **稳定**，不是指一句话永远不改，而是指：

- 不同信息各归各位
- 变化发生时，影响边界可控
- 新功能加进来，不会把旧行为全部打乱

所以这个项目里：

- `systemPromptSection()` 是稳定性设计
- `dynamic boundary` 是稳定性设计
- `attachment` 也是稳定性设计

这是一种非常重要的认知变化：

Prompt Engineering 到了系统阶段，重点从“写出神 prompt”转向“建立 prompt 结构稳定性”。

4.8 Prompt 为什么要和缓存策略一起设计

这是很多人会忽略的点。

在小项目里，Prompt 只是行为问题。

但在长会话、多工具、多 Agent 系统里，Prompt 同时也是：

- 成本问题
- 延迟问题
- 规模问题

这就是为什么 Claude Code 的 Prompt 设计会强绑定：

- `cache`
- `dynamic boundary`
- `uncached section`

如果以后你自己设计系统，记住一个非常实用的原则：

- 不要先问“这段提示该怎么写”
- 先问“这段提示会多久变化一次”

因为变化频率，决定它该不该放进缓存友好的前缀里。

4.9 如果你自己设计类似系统，Prompt 这一层该怎么选

这里我给你一个很实用的分层建议。

第一层：永远稳定的规则

例如：

- 角色定义
- 安全底线
- 基础工具使用原则

放这里的内容要尽量少变。

第二层：模式相关规则

例如：

- 当前是 coordinator 模式
- 当前是某个自定义 agent
- 当前有特殊输出风格

这一层适合通过组合函数统一管理优先级。

第三层：本轮动态材料

例如：

- 新发现的技能
- 新连接的 MCP instructions
- relevant memories
- plan mode exit 提醒

这一层更适合 attachment，而不是重写整段 system prompt。

如果你一开始就按这三层设计，后面很多痛苦会少很多。

4.10 这一节你真正该学到的东西

如果把 Prompt 这一节压成一句话，就是：

好的 Prompt 系统，不是把所有信息都写进去，而是把不同稳定度、不同职责的信息放到正确的位置。

这比“某一句 prompt 写得多漂亮”重要得多。

5. Tool Calling：为什么这里不是简单 Function Calling，而是一套工具运行时

很多教程讲 `Function Calling`，都讲得很轻：

1. 给模型一个函数 schema
2. 模型返回函数名和参数
3. 程序执行

但真实系统一复杂，这种理解就远远不够。

5.1 `Tool.ts` 在做什么

对应位置：

- `restored-src/src/Tool.ts`

这里最重要的不是某一个工具，而是 `ToolUseContext`。

`Context` 这个词你可以理解成：

- 工具在执行时所处的环境

它里边不仅有工具列表，还有：

- 当前命令列表
- MCP 客户端
- 当前 app state
- 中断控制器
- 读文件缓存
- 通知能力
- Agent 身份
- 当前消息集合
- 当前权限上下文

为什么这个设计很关键？

因为工具执行从来不只是“给我一个参数，我返回一个结果”。

真实工具经常要知道：

- 我现在是不是子 Agent
- 我有没有权限写这个路径
- 当前会话里还有哪些工具
- 当前是否允许弹交互界面

如果这些都不统一进上下文，后面系统一定会乱。

5.2 `toolOrchestration.ts` 在做什么

对应位置：

- `restored-src/src/services/tools/toolOrchestration.ts`

这个文件主要解决一个问题：

- 一轮 assistant message 里如果有多个 tool call，到底怎么执行？

这里有个重要术语：

- `Concurrency-safe`：并发安全

通俗解释：

- 几个工具能不能同时跑，而不会互相打架

例如：

- 同时读多个文件：通常没问题
- 同时写同一个文件：通常有风险

所以这里不是简单地：

- 全部串行
- 或全部并行

而是先分批：

- 并发安全的，一起跑
- 不安全的，一个个跑

为什么这种做法好？

因为它同时兼顾了：

- 性能
- 正确性

如果全串行，慢。

如果全并行，乱。

5.3 权限层为什么这么重要

很多人一开始会觉得：

- “模型自己会决定要不要调工具，不就够了吗？”

不够。

因为模型不等于安全系统。

你必须再回答：

- 这个工具有没有风险
- 用户是否允许
- 子 Agent 是否能替主 Agent 请求授权

这里的 allow / deny / ask 体系，本质上是在做一层“人机协作安全闸门”。

尤其一旦进入：

- shell
- 文件写入
- 外部系统 API
- 多 Agent

权限系统就不是增强项，而是底盘。

5.4 为什么 MCP 重要

对应位置：

- `restored-src/src/services/mcp/client.ts`

MCP 你可以把它理解成：

- 一个统一的外部能力接入协议

不是每个外部工具都要你自己硬编码。

通过 MCP，外部服务可以带着：

- tools
- resources
- prompts
- instructions

接入 Agent 系统。

这意味着系统不再只是本地工具集合，而是一个可扩展能力平台。

为什么这很关键？

因为一旦到了真实企业环境，Agent 的价值往往不在：

- “会不会读文件”

而在：

- “能不能连 GitHub、Slack、Jira、浏览器、内部知识系统”

5.5 Tool Search 在解决什么问题

对应位置：

- `restored-src/src/utils/toolSearch.ts`

这个模块解决的是一个很现实的问题：

- 工具越来越多怎么办？

如果你把几十个、上百个工具的完整描述都提前塞进上下文，会带来两个后果：

- 上下文非常贵
- 模型选择也会变差

所以这里引入了：

- `Deferred loading`：延迟加载

意思是：

- 先不把所有工具完整暴露给模型
- 需要时再通过 ToolSearch 去发现相关工具

这其实已经很像一种“能力检索（Capability Retrieval）”。

也就是说，它不只是：

- 检索文档

它还在：

- 检索可用动作

这是一种非常前沿、也非常实用的 Agent 工程思路。

5.6 如果不这样做，会有什么更“简单”的方案

为了让你真正理解这里为什么复杂，我把几种更简单的替代方案也摆出来。

方案 A：所有工具一开始全量暴露给模型

这在早期很常见，做法就是：

- 启动时把所有工具 schema 和 description 一次性发给模型
- 后面每轮都继续带上

优点：

- 实现最简单
- 模型一开始就知道“自己都能做什么”

缺点：

- 工具一多，上下文成本飙升
- 模型会被过多工具分散注意力
- MCP 外部工具一接多，效果会越来越差

Claude Code 为什么没有走这条路走到底？

因为它已经进入了“工具生态不断增长”的阶段。这个阶段的核心问题不再是“有没有工具”，而是“工具太多了，怎么让模型只看到该看到的那部分”。

方案 B：所有工具全串行

做法就是：

- 模型发出多个 tool call
- 系统一个个排队执行

优点：

- 非常稳
- 调试简单

缺点：

- 太慢
- 读操作也被迫排队
- 大量本来可以并行的 I/O 被浪费

Claude Code 的改进就在于：

- 它不盲目并行
- 也不盲目串行
- 而是做“并发安全分批”

这是典型的工程折中，不追求绝对理论优雅，而追求在现实里又快又稳。

方案 C：工具执行后直接把结果塞给业务逻辑，不回到消息流

这也是不少系统的早期做法：

- 工具返回一个对象
- 程序自己用这个对象判断下一步

看起来好像更“工程化”，但它有个很大的问题：

- 模型就失去了对工具结果的自然推理链路

Claude Code 这里的核心选择是：

- 工具结果回写成消息
- 再进入 query loop

这让模型可以继续基于真实世界反馈调整行动。

这本质上保留了 Agent 最重要的一点：

- 决策不是写死在代码里的
- 决策依然在模型和环境的循环里发生

5.7 Tool Calling 真正的难点，不在调用，在“副作用治理”

这里再往深处讲一层。

很多人以为 Tool Calling 的难点是：

- 怎么把 JSON 参数解析对

其实真实难点往往是：

- 工具执行以后，会对世界造成什么副作用

所谓 **副作用**，你可以理解成：

- 不是只读结果
- 而是改变了文件、网络状态、任务状态、权限状态

例如：

- `read_file` 基本没有副作用
- `edit_file` 有文件副作用
- `bash` 既可能只读，也可能极具破坏性
- `MCP tool` 可能改远端系统

所以你现在可以更好理解为什么这个项目会引入：

- permission system
- concurrency-safe
- canUseTool
- interruptBehavior
- leader/worker permission bridge

这些都不是“加戏”，而是在治理副作用。

如果不治理副作用，会发生什么？

- 模型乱改文件
- 子 Agent 越权操作
- 多工具并发导致状态错乱
- 用户根本不敢信任系统

5.8 这一节你真正该学到的东西

如果把 Tool Calling 这一节压缩成一句工程经验，就是：

当工具开始接触真实世界时，Tool Calling 就不再是“函数调用”，而是“副作用受控的动作执行系统”。

这句话很重要。你以后看任何 Agent 系统的工具层，都可以用这句话去判断它成熟不成熟。

- 如果它只会展示 schema，那还很早期
- 如果它开始讨论权限、并发、中断、缓存、回流，那它就开始进入工业阶段了

6. Workflow 编排：你刚才举的那段为什么会显得“干巴巴”，现在我把它讲透

你举的那段原文是：

- 单轮 query loop
- 会话级编排
- 背景任务编排
- 多 Agent 编排

如果只是这样列出来，确实会很干，因为它没有讲清楚：

- “编排”的对象是什么
- 每一层为什么要单独存在
- 真正是怎么实现的

下面我用“讲稿”方式重新讲一遍。

6.1 Workflow 到底是什么意思

这里的 workflow 不要理解成传统 BPM 流程图，也不要只理解成代码函数调用顺序。

这里的 Workflow 更准确地说，是：

一个任务从开始到结束，在哪些阶段发生了什么、由谁负责、靠什么机制往下流动。

也就是说，Workflow 关心的是：

- 任务不是静止的
- 任务在流动

6.2 第一层：单轮 query loop 编排

这里的流程是：

- 模型 -> tool use -> tool result -> 再推理

如果要把它讲白一点，就是：

1. 模型先看当前上下文，判断“我下一步要做什么”
2. 如果它觉得要查文件、跑命令、搜资源，它不会直接瞎回答，而是发一个 `tool use`
3. 系统收到这个 `tool use` 后，真正去执行动作
4. 执行结果回来以后，系统不是直接扔给用户，而是先喂回模型
5. 模型基于新结果继续判断，决定下一步是继续调工具，还是给最终结论

这就是单轮 query loop 编排。

为什么这一层必须存在

因为大多数真实任务都不是一步能答完的。

比如用户说：

- “帮我看看为什么测试挂了”

模型如果不先：

- 查测试文件
- 跑命令
- 读报错

它很可能只是空谈。

所以这一层的本质是：

- 把“思考”和“行动”交替组织起来

真正是怎么实现的

核心在：

- `restored-src/src/query.ts`

你可以把这个文件理解成“主控循环”。

它做的事情包括：

- 构造 Prompt 和上下文
- 发起模型请求
- 读 assistant 消息
- 发现 `tool_use`
- 调用 `runTools()`
- 把 `tool_result` 再写回消息流
- 继续下一轮推理

这一层的亮点

亮点不只是“能跑工具”，而是它考虑了很多现实问题：

- 工具能不能并发
- token 超了怎么办
- 输出太长怎么办
- 中途中断怎么办
- 失败后怎么恢复

所以它不是一个 demo loop，而是一个可持续工作的 loop。

6.3 第二层：会话级编排

你原文里的列法是：

- auto compact
- stop hooks
- post compact cleanup
- session restore

我现在把它翻成更好理解的话。

6.3.1 为什么会有“会话级编排”

因为单轮 query loop 只关心：

- 这一轮怎么做完

但真实系统里还要关心：

- 这一场会话怎么活下去

会话级编排关心的是“生命周期”，不是“当前一步”。

这就像项目管理里：

- 单轮 loop 是“今天干什么”
- 会话级编排是“这个项目接下来几周怎么不断下去”

6.3.2 auto compact 是什么

`Compact` 我前面解释过，就是“上下文压缩”。

为什么需要它？

因为会话一长，历史消息会越来越多，而模型上下文窗口不是无限的。

所以系统会在适当时候做：

- 自动摘要
- 用摘要替代旧历史

这就是 `auto compact`。

为什么不让用户手动控制就好？

因为真实用户不会一直盯着 token 使用情况。系统必须自己具备“自我瘦身”能力，不然长会话迟早死掉。

对应实现：

- `restored-src/src/services/compact/autoCompact.ts`
- `restored-src/src/services/compact/compact.ts`

6.3.3 stop hooks 是什么

Hook 你可以理解成“钩子”。

在软件系统里，钩子就是：

- 某个事件发生后，自动触发的一段额外逻辑

stop hooks 的意思是：

- 当前这轮模型采样结束后，要不要顺手做一些后处理

比如：

- 提取 memories
- 触发 auto dream
- 做一些会话后整理

也就是说，它不是主任务的一部分，但它和主任务结束强相关，所以放在 stop hooks 里。

对应实现：

- `restored-src/src/query/stopHooks.ts`

6.3.4 post compact cleanup 是什么

这个词也容易显得干巴。

简单说就是：

- compact 不是把历史压完就结束
- 压完以后还要“清理和重置一些状态”

为什么？

因为 compact 之后，很多缓存、附件状态、技能发现状态可能已经不可靠了。

如果不清理，就会出现：

- 该重新注入的没注入
- 不该保留的旧状态还在

对应实现：

- `restored-src/src/services/compact/postCompactCleanup.ts`

6.3.5 session restore 是什么

意思是：

- 用户下次回来，或者中途恢复时，怎么把这场会话接上

这不是简单读取聊天记录，而是要恢复：

- 当前历史
- compact 边界
- 会话模式
- 必要的运行状态

对应实现分布在：

- `utils/sessionStorage.ts`
- `utils/sessionRestore.ts`
- `assistant/sessionHistory.ts`

会话级编排这一层的核心亮点

这一层真正厉害的地方是：

- 它开始从“完成一轮任务”升级到“维持一个长期工作会话”

很多系统能做一两轮任务，但一到几十轮就明显失真、变慢、失忆。Claude Code 明显就是在专门解决这个问题。

6.4 第三层：背景任务编排

你原文里的列法是：

- extract memories
- auto dream
- summaries

现在我把它讲成“系统背后在悄悄做什么”。

6.4.1 为什么要有背景任务

因为有些工作很重要，但不适合堵在主线程里。

比如：

- 把本次会话里值得长期保存的信息提出来
- 把多次会话里的 memory 合并整理
- 生成额外摘要

这些事情如果全放在用户当下这一轮里，会带来问题：

- 响应更慢
- 主 Agent 注意力被打断
- 用户并不一定需要立刻看到这些内部过程

所以系统把它们做成“后台编排”。

6.4.2 `extract memories` 到底是什么

它不是“模型顺手记一下”那么简单。

它更像：

- 当前会话结束以后，系统开一个专门的后处理 Agent
- 回顾这次会话
- 找出真正值得持久保存的内容
- 写到 memory 目录里

为什么这么做更好？

因为主 Agent 在解决主问题时，不一定总能稳定、主动地做好记忆抽取。

把这件事后置，有两个好处：

- 抽取更系统
- 不干扰主任务

对应实现：

- `restored-src/src/services/extractMemories/extractMemories.ts`

这里还有一个很漂亮的设计点：

- 它会限制这个 extraction agent 的工具权限
- 只允许读工具，和在 memory 路径范围内写

这说明作者不只是想到“开个子 Agent 去提取记忆”，还想到：

- 这个子 Agent 不应该有过大权限

6.4.3 `auto dream` 到底是什么

这个名字有点诗意，但本质上是：

- 系统在更长时间尺度上做 memory consolidation（记忆整理与归并）

为什么需要它？

因为仅仅有 `extract memories` 还不够。

如果你每次都抽取一堆新的 memory，但从不整理，最后会出现：

- memory 越来越多
- 重复越来越多
- 索引越来越乱

所以 `auto dream` 是在做更长周期的记忆归并。

你可以把它理解成：

- `extract memories`：当天随手记笔记

- auto dream: 过几天整理知识卡片

对应实现:

- `restored-src/src/services/autoDream/autoDream.ts`

6.4.4 summaries 指什么

这里的 summaries 不是一个固定模块名, 而是泛指系统里那些“为了后续更好工作而生成的摘要类产物”。

包括:

- compact summary
- session memory summary
- background summary

摘要在这个系统里不是为了用户读, 而主要是为了系统自己继续工作。

这点非常重要。

6.5 第四层: 多 Agent 编排

你原文里的列法是:

- coordinator -> workers
- swarm teammates
- mailbox / notification / task state

这里如果不解释, 确实最容易看不懂。因为它混合了角色、运行方式、基础设施三类概念。

我把它拆开讲。

6.5.1 coordinator -> workers 是角色编排

这说的是:

- 主协调 Agent 负责理解任务、拆任务、收结果
- worker 负责局部执行

这是一种角色分工, 不是一种底层通信协议。

对应实现和规则主要在:

- `restored-src/src/coordinator/coordinatorMode.ts`

6.5.2 swarm teammates 是运行机制

这说的是:

- 多个 Agent 以团队方式运行
- 它们之间有上下文边界、权限桥接、状态管理

这是一种“多执行体系统”的基础设施, 不只是 Prompt。

对应实现:

- `restored-src/src/utis/swarm/*`

6.5.3 mailbox / notification / task state 是结果回流机制

这是最底层、也是最工程的一层。

意思是：

- 子 Agent 不是在真空中工作
- 它完成、失败、请求权限、上报进度，都要有统一通道

这里：

- `mailbox` 可以理解成“消息收发箱”
- `notification` 可以理解成“状态通知”
- `task state` 可以理解成“任务状态对象”

为什么一定要有这层？

因为如果没有统一回流机制，多 Agent 只会变成多个孤立会话，最后主线程根本无法可靠综合。

对应实现：

- `utils/teammateMailbox.ts`
- `utils/task/framework.ts`
- `tasks/*`
- `utils/swarm/inProcessRunner.ts`

6.6 这一整套 Workflow 编排到底最值得学什么

最值得学的不是某个模块名，而是一个思想：

编排不是把步骤列出来，而是把“不同时间尺度、不同执行体、不同责任边界”的工作流拆开管理。

在这个项目里：

- 单轮 query loop 负责当前一步怎么推进
- 会话级编排负责整场会话怎么活下去
- 背景编排负责长期价值怎么沉淀
- 多 Agent 编排负责复杂任务怎么拆分与协作

这就是为什么它不像“一个模型加一堆工具”，而更像一个真正的工作系统。

6.7 如果不做分层 Workflow，会落入哪些常见陷阱

这一节也值得再补透一点。

陷阱 A：把所有流程都塞进 query loop

早期系统经常这么做：

- 什么都在主循环里完成
- 工具执行在里面
- memory 提取也在里面

- compact 也在里面
- 后处理也在里面

短期好像很省事，但后面会怎样？

- query loop 越来越胖
- 任何小改动都可能影响全局
- 调试和恢复会越来越难

Claude Code 的做法不是让 query loop 什么都不管，而是：

- query loop 负责主过程
- stop hooks、background tasks、compact system 分别接管不同后续环节

陷阱 B：把会话级问题当成单轮问题

这是另一个很常见的问题。

比如：

- 会话太长
- 恢复失败
- compact 后状态不一致

这些都不是单轮问题，但很多系统会试图在“当前请求逻辑”里顺手补丁解决。

这样做通常会造成：

- 一堆 if/else
- 逻辑越补越乱

Claude Code 更成熟的地方在于，它承认这些是“会话生命周期问题”，所以用：

- `auto compact`
- `post compact cleanup`
- `session restore`

单独治理。

陷阱 C：把后台沉淀任务强塞进用户响应路径

比如：

- 用户问一个问题
- 系统当场还顺手做 memory extraction、summary、知识归档

这会带来：

- 响应变慢
- 主任务被打断
- 用户不知道系统在忙什么

所以这里的背景任务编排，真正价值就是：

- 把“现在必须做的”和“稍后值得做的”分开

陷阱 D：多 Agent 只有 spawn，没有回流协议

这是最危险的一种。

很多系统能做到：

- 启动多个子 Agent

但做不到：

- 明确通知谁完成了
- 失败时怎么继续
- 权限请求怎么上返
- coordinator 怎么收敛结果

结果就是：

- 看起来像多 Agent
- 实际上像多个散乱脚本

Claude Code 在这一层真正补的是“任务协议”。

6.8 Workflow 真正难的地方，不是“画流程图”，而是“时间尺度管理”

这里我觉得特别值得讲透。

很多人一说 workflow，就会想：

- 第一步做什么
- 第二步做什么
- 第三步做什么

但 Claude Code 这种系统真正难的是：

- 不同事情发生在不同时间尺度上

例如：

- `query loop`：秒级
- `compact`：会话中期
- `extract memories`：某轮结束后
- `autoDream`：更长周期
- `coordinator/worker`：并行任务周期

如果你不按时间尺度拆，所有东西都会混成一锅。

这也是为什么原来那句“workflow 至少发生在四个层次”虽然对，但如果不展开讲，就很难真的理解。

更准确的说法应该是：

这个系统把任务流动拆成了四个时间尺度的工作流，每个尺度解决的问题都不同。

6.9 如果你自己设计类似系统，Workflow 这一层怎么选

你可以按下面这个判断方式来做。

如果你还在做 MVP

只需要先做：

- 单轮 query loop

先别做：

- 多层 workflow
- 背景任务
- 多 Agent 编排

因为主循环不稳时，所有额外流程都是放大混乱。

如果你的会话开始变长

就要加：

- compact
- session restore

也就是进入“会话级编排”。

如果你开始需要长期价值沉淀

就要加：

- stop hooks
- memory extraction
- background summary

如果你开始做复杂任务拆分

才要加：

- coordinator
- task notifications
- mailbox / result return channel

也就是说，workflow 编排也不是一开始就要全做，而是跟着复杂度自然长出来。

6.10 这一节你真正该学到的东西

如果把 Workflow 这一节压成一句话，就是：

Workflow 的本质不是把步骤列出来，而是按时间尺度和责任边界，把任务流动分层管理。

这是 Claude Code 在工程上非常成熟的一点，也是很多只会写“Agent demo”的系统最缺的部分。

7. RAG 检索增强：它不是典型向量库 RAG，但绝对不是没有 RAG

很多人一听 RAG，就自动想到：

- 文档切块
- embeddings
- vector database

但这其实只是 RAG 的一种实现路线，不是唯一形态。

在这个项目里，RAG 更像“混合检索增强”。

7.1 什么叫 RAG

先用最朴素的话说：

`RAG = 先找，再答`

也就是：

- 模型不是完全只靠自己参数记忆作答
- 它会先去外部或历史资料里找相关信息
- 再基于这些信息生成答案

7.2 这个项目里的第一类 RAG：规则型检索

对应位置：

- `restored-src/src/utils/claudemd.ts`

这里的检索对象不是普通文档，而是：

- `CLAUDE.md`
- `.claude/rules/*.md`
- 用户级规则
- 项目级规则
- 本地规则

为什么这也是 RAG？

因为系统是在：

- 先找与当前工作目录、层级、路径匹配的规则文件
- 再把这些规则注入上下文

这就是一种“规则检索增强”。

它特别适合：

- 工程规范
- 项目约定
- 团队规则

7.3 第二类 RAG：相关记忆检索

对应位置：

- `restored-src/src/memdir/findRelevantMemories.ts`

这里的做法很有意思：

1. 先扫描 memory 文件头
2. 拼成 manifest
3. 用 sideQuery 让一个小模型来选出最相关的 memory
4. 再把它们注入当前上下文

这意味着它不是：

- 直接把所有 memory 全喂给主模型

而是：

- 先筛选
- 再注入

这里的 `selector`（选择器）其实在扮演一个“轻量 reranker（重排器）”角色。

7.4 第三类 RAG：工具检索

对应位置：

- `restored-src/src/utis/toolSearch.ts`

这个很值得学，因为它说明：

- 不只是文档可以检索
- 工具能力本身也可以检索

当工具很多时，系统不会全量暴露，而是：

- 先延迟加载
- 需要时再搜索相关工具

这其实是一种“能力检索增强”。

你可以把它理解成：

- 普通 RAG：找信息
- Tool RAG：找能做这件事的工具

7.5 第四类 RAG：外部资源检索

对应位置：

- `services/mcp/client.ts`
- `tools/webFetchTool/*`

这里系统还支持：

- 把外部 MCP 资源拉进来
- 把网页内容抓进来
- 再交给模型做进一步处理

这说明它的检索增强不是局限在本地记忆，而是可以接外部世界。

7.6 为什么它现在不一定需要重型向量库

从当前源码看，没有明显的主干向量数据库实现。

这未必是缺点。

因为很多系统在早中期，用下面这几种方式就能解决大部分问题：

- 层级规则文件
- 文件型 memory
- 小模型选择器
- 外部资源按需抓取
- 工具延迟暴露

只有在这些场景变重时，向量库才会明显值得：

- 记忆量特别大
- 跨项目语义搜索很强
- 需要高速大规模召回

这对你有个很重要的启发：

RAG 不等于向量库，真正应该先问的是：我要检索的到底是什么。

7.7 为什么很多团队会“过早向量化”

这里我想专门提醒一个非常常见的误区。

很多团队一做知识增强，就会下意识地走这条路线：

1. 所有文档切 chunk（切片）
2. 全部做 embedding（向量化）
3. 扔进 vector DB（向量数据库）
4. 检索 top-k

这条路线不是错，而是很容易“太早”。

为什么太早？

因为它默认了三个前提：

1. 你的知识主要是自由文本
2. 你的问题主要是语义匹配问题
3. 你已经有足够大规模的数据，值得承担索引和治理成本

但 Claude Code 这个场景里，很多“知识”其实不是自由文本，而是：

- 规则
- 约定
- 偏好
- 路径作用域
- 工具能力说明

这些内容很多时候不适合简单切块向量化。

比如：

- “在 `src/payments/**` 目录下遵守某条规则”
- “这个用户不喜欢长解释”
- “这个 Agent 不能用某些工具”

这些东西更适合：

- 文件组织
- frontmatter
- 结构化元数据
- 精准路径匹配

而不是一股脑做 embedding。

7.8 这个项目的 RAG 更像“混合检索”，不是“单一检索”

你可以把它的检索方式理解成四种手段混用：

第一种：规则命中

例如：

- 按目录层级查 `CLAUDE.md`
- 按路径作用域查规则文件

这是结构化命中，不是语义搜索。

第二种：元数据筛选 + 小模型选择

例如：

- `findRelevantMemories.ts`

它先扫描 memory 头部元数据，再让模型做“是否相关”的选择。

这是“轻量语义筛选”，但不是大规模向量检索。

第三种：能力检索

例如：

- `Tool Search`

它检索的不是知识文本，而是“系统当前有哪些动作能力”。

第四种：外部资源按需拉取

例如：

- MCP resources
- WebFetch

这是在线检索，不是预索引检索。

所以如果你以后自己设计 RAG，不要先问：

- 我要不要上向量库？

而应该先问：

- 我的知识到底分几类？
- 每类知识最适合哪种检索方式？

这是从“工具导向”转向“问题导向”的关键一步。

7.9 如果未来要升级成更重型 RAG，应该怎么演进

假设你的系统以后变得更大，什么时候真的需要更强 RAG？

通常会在这些场景：

- 项目数很多
- 知识量很大
- 团队共享知识越来越多
- 语义搜索需求越来越强

这时更合理的演进方式不是推翻现有体系，而是叠加一层：

1. 保留规则型文件系统
2. 保留 memory 文件与索引
3. 额外增加语义索引层
4. 在召回时做混合排序

也就是：

- 路径/规则命中继续保留
- 自由文本知识再上 embedding

这会比“一开始就全量向量化”稳得多。

7.10 这一节你真正该学到的东西

如果把这一节再压成一句话，就是：

RAG 的核心不是“上某种库”，而是“按知识类型选择合适的取回方式”。

Claude Code 最值得学的地方，不是它用了多新潮的检索技术，而是它很清楚：

- 规则型知识怎么取

- 记忆型知识怎么取
- 能力型知识怎么取
- 外部知识怎么取

8. 会话记忆：为什么它不是一个 memory store，而是多层记忆体系

如果只看“记忆”两个字，很容易觉得：

- 存下来不就行了？

但真实系统里，最难的不是“存”，而是“分层”。

8.1 第一层：短期工作记忆

就是当前 `messages`。

作用：

- 支撑当前任务
- 让模型记住刚刚发生了什么

问题：

- 很快就会膨胀
- 混有很多临时噪声

所以它不能直接当长期 memory。

8.2 第二层：会话级压缩记忆

对应位置：

- `services/compact/*`
- `services/compact/sessionMemoryCompact.ts`

这一层的作用是：

- 让会话在很长的时间还能继续

注意，这一层不是为了“未来跨会话复用”，而主要是为了：

- 这一场会话别死

这是很多系统容易混淆的地方。

8.3 第三层：持久记忆

对应位置：

- `restored-src/src/memdir/memdir.ts`
- `teamMemPrompts.ts`

这里用的是文件型 memory:

- 每条记忆单独一个文件
- 有 frontmatter
- 有 `MEMORY.md` 索引

为什么这么做?

因为它更适合:

- 人工编辑
- Agent 自己读写
- Git 追踪
- 审计和去重

8.4 为什么还要 memory taxonomy (记忆类型体系)

因为如果不区分类型, 模型很容易把:

- 当前临时任务计划
- 长期用户偏好
- 项目事实

全部混在一起记。

结果就是:

- 记忆越来越多
- 但可用性越来越差

所以这个项目很强调:

- 什么该记
- 什么不该记
- 什么应该进 plan, 而不是进 memory

这本质上是在做“记忆治理”。

8.5 `extractMemories` 的真正意义

对应位置:

- `services/extractMemories/extractMemories.ts`

它的意义不是“帮忙多存点东西”, 而是:

- 主线程不一定最适合做长期记忆抽取

所以系统会在 stop hooks 之后, 用一个更专注的流程去看:

- 这次会话里, 什么东西值得以后再记得

这很像一个人工作结束后复盘, 而不是边干边整理人生经验。

8.6 autoDream 的真正意义

对应位置：

- `services/autoDream/autoDream.ts`

它解决的问题不是“没有记忆”，而是：

- 记忆越来越碎怎么办

如果把 `extractMemories` 比作“当天写笔记”，那 `autoDream` 更像“定期做知识整理”。

这一步很关键，因为没有它，memory 系统迟早会从资产变成噪声。

8.7 为什么记忆系统最难的不是“存”，而是“边界”

这是我非常想强调的一点。

很多人做 memory 时，天然会把重点放在：

- 怎么存得更多
- 怎么检索得更快

但真实最难的问题其实是：

- 什么该存
- 什么不该存

这就是 `边界问题`。

例如下面这些内容，很容易混在一起：

- 用户长期偏好
- 某个项目的稳定事实
- 当前会话临时计划
- 某次失败后的临时观察

如果都扔进长期 memory，会出什么问题？

- 模型以后会把临时信息当长期事实
- 噪声会越来越多
- 记忆可信度会下降

这也是为什么 `memdir/memdir.ts` 和 `teamMemPrompts.ts` 要反复强调：

- 什么时候用 memory
- 什么时候用 tasks
- 什么时候用 plan

本质上，这是在做“记忆边界治理”。

8.8 为什么这个项目采用“文件 + 索引”的 memory 结构

这里再讲细一点。

它不是只做一堆 memory 文件，还专门引入了：

- 每条记忆一个文件
- `MEMORY.md` 作为入口索引

为什么不直接做一个大文件？

如果只用一个大文件，问题会很快出现：

- 越写越长
- 很难维护主题边界
- 很难更新局部内容
- 很难去重

为什么不用数据库表直接存？

那样当然也可以，但会带来另一些成本：

- Agent 自己不如写文件自然
- 用户和开发者不如直接改 Markdown 方便
- 不如 Git diff 和审阅直观

所以这里的设计其实是在平衡三件事：

1. Agent 易写
2. 人类易读
3. 结构可治理

文件 + 索引是一个很实用的中间解。

8.9 `extractMemories` 和 `autoDream` 解决的是不同时间尺度的问题

为了让这个体系更清楚，你可以把它们放到不同时间尺度上理解。

`extractMemories`

时间尺度：

- 当前会话结束附近

目标：

- 把刚刚发生的、有长期价值的信息提出来

`autoDream`

时间尺度：

- 更长周期，例如多次会话之后

目标：

- 回过头看这些 memory 是否应该合并、重写、精简

这其实对应两种完全不同的工作：

- 抽取
- 整理

如果你只做抽取不做整理，会怎样？

- memory 数量会涨
- 但质量会掉

如果你只做整理不做抽取，会怎样？

- 根本没有足够素材可以整理

所以这两步必须是配套的。

8.10 记忆系统在哪些新场景下会暴露不足

当前这种文件型、多层记忆体系非常适合：

- 单人或小团队
- 项目式工作
- 代码协作场景

但如果往更大场景走，也会出现一些边界：

场景 A：知识量急剧增大

问题：

- 文件扫描会变慢
- relevance selection 成本升高

场景 B：跨项目、跨组织共享

问题：

- 权限治理会变难
- 哪些 memory 该共享、哪些不该共享，需要更强的策略层

场景 C：记忆事实需要强一致

问题：

- 文件型 memory 更偏实用与灵活
- 不适合当成强一致数据库

也就是说，这套记忆系统很强，但它更像“Agent 知识工作台”，还不是“企业主数据平台”。

8.11 这一节你真正该学到的东西

如果把这一节压成一句话，就是：

好的 memory 系统，不是把更多历史留下来，而是把真正会在未来继续有价值的东西，以合适层级留下来。

这也是为什么 Claude Code 的 memory 系统看上去有点“麻烦”，因为它不是在追求“能记”，而是在追求“记得住、记得对、还能继续用”。

9. 工业知识库构建：这个项目走到了哪一步，还差什么

这一部分很多人会混淆“记忆”和“知识库”。

我先给你一个区分：

- `Memory`：偏向某次会话、某个用户、某个项目积累出来的经验和偏好
- `Knowledge Base`：更偏向结构化、可复用、可治理的知识底座

9.1 这个项目已经有知识库雏形

它的知识载体包括：

- `CLAUDE.md`
- `.claude/rules/*.md`
- `memory/*.md`
- `MEMORY.md`
- team memory
- skills
- MCP resources
- agent definitions

注意这里很重要的一点：

这些知识不是静态摆在那里，而是会直接进入运行时，影响 Agent 行为。

所以它更像：

- 可执行知识库

不是单纯文档中心。

9.2 为什么文件型知识库对代码 Agent 特别有价值

因为它和代码世界天然兼容：

- 都是文件
- 都能走 Git
- 都能 diff
- 都能让人和 Agent 共编

这比纯数据库路线更容易早期落地。

9.3 它目前还差什么，才算真正企业级知识中台

如果继续演进，通常还需要：

- 统一 knowledge provider 抽象
- 混合检索能力
- 权限治理
- 来源与可信度标记
- 版本和冲突管理
- 评估体系

也就是说，它已经很像“知识底座的雏形”，但还没有完全成为“企业知识中台”。

10. 最后总结：你真正应该从这份源码里学到什么

如果让我把这份项目的启发压成几句话，我会这么说。

10.1 不要把 Agent 理解成“大模型会调工具”

更准确的理解应该是：

- Agent = 模型 + query loop + tool runtime + context system + memory system

10.2 不要把 Prompt 理解成“一段提示词”

更准确的理解应该是：

- Prompt = 稳定规则 + 动态上下文 + 附件注入 + 优先级系统

10.3 不要把 Workflow 理解成“几个步骤”

更准确的理解应该是：

- Workflow = 任务在不同时间尺度、不同责任边界中的流动方式

10.4 不要把 Memory 理解成“多存点历史”

更准确的理解应该是：

- Memory = 对未来有价值的信息，被正确分类、正确抽取、正确整理、正确找回

10.5 不要把多 Agent 理解成“多开几个模型”

更准确的理解应该是：

- Multi-Agent = 角色分工 + 任务协议 + 权限桥接 + 状态回流 + 综合决策

Claude Code 这套系统真正成熟的地方，不在某个单点技巧，而在于它很早就承认：

一旦模型开始长期工作，复杂性不会消失，只能被分层管理。

这就是这份源码最值得你学的东西。

11. 关键源码索引

- `restored-src/src/main.tsx`
- `restored-src/src/QueryEngine.ts`
- `restored-src/src/query.ts`
- `restored-src/src/constants/prompts.ts`
- `restored-src/src/utils/systemPrompt.ts`
- `restored-src/src/constants/systemPromptSections.ts`
- `restored-src/src/utils/queryContext.ts`
- `restored-src/src/context.ts`
- `restored-src/src/utils/attachments.ts`
- `restored-src/src/Tool.ts`
- `restored-src/src/services/tools/toolOrchestration.ts`
- `restored-src/src/services/mcp/client.ts`
- `restored-src/src/services/compact/compact.ts`
- `restored-src/src/services/compact/sessionMemoryCompact.ts`
- `restored-src/src/query/stopHooks.ts`
- `restored-src/src/memdir/memdir.ts`
- `restored-src/src/memdir/findRelevantMemories.ts`
- `restored-src/src/utils/claudemd.ts`
- `restored-src/src/services/extractMemories/extractMemories.ts`
- `restored-src/src/services/autoDream/autoDream.ts`
- `restored-src/src/tools/AgentTool/loadAgentsDir.ts`
- `restored-src/src/tools/AgentTool/runAgent.ts`
- `restored-src/src/tools/AgentTool/prompt.ts`
- `restored-src/src/coordinator/coordinatorMode.ts`
- `restored-src/src/utils/swarm/inProcessRunner.ts`
- `restored-src/src/utils/forkedAgent.ts`